

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1 - INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	60%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	Raiyan I	Reg.No.:	192521421
Department:	Btech IT	Date:	26/08/2026

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

- Smartphone Performance Optimization**
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I G Devendra Nath Reddy(192425204) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem(20%)	Application of Engineering Concepts(25%)	Solution Design(25%)	Use of Tools(15%)	Analysis(10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1.Smartphone Performance Optimization

Problem

A mobile manufacturer observes lag while switching between applications. The main reason can be delays caused by accessing data from slower memory when the required data is not available in the processor cache.

Role of L1, L2, L3 Cache and DRAM

Memory	Speed	Size	Main Role
L1 Cache	Fastest	Smallest	Stores immediately required instructions and data
L2 Cache	Very Fast	Larger than L1	Stores frequently used data not found in L1
L3 Cache	Fast	Much larger	Shared cache that reduces access to main memory
DRAM	Slower	Very Large	Stores running applications and operating system data

Analysis

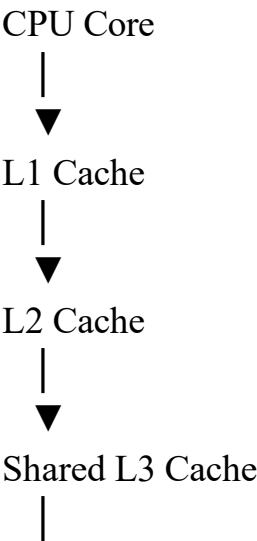
When switching between apps:

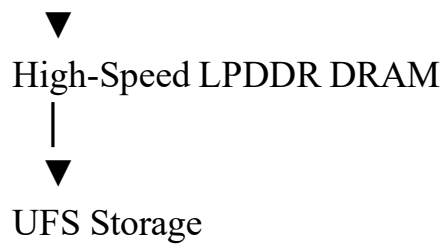
1. The processor first checks **L1 cache**.
2. If data is unavailable, it checks **L2 cache**.
3. If still unavailable, it accesses **L3 cache**.
4. Finally, it accesses **DRAM**, which has significantly higher latency.

Frequent DRAM accesses can cause delays during application switching.

Proposed Hierarchical Memory Redesign

Recommended Structure:





Improvements

- Increase **L2/L3 cache capacity** to reduce cache misses.
- Use efficient **shared L3 cache** for multiple CPU cores.
- Use high-bandwidth, low-power **LPDDR memory**.
- Improve cache replacement and prefetching mechanisms.
- Keep recently used application data in DRAM instead of immediately removing it.

Result

The redesigned hierarchy reduces expensive DRAM accesses, resulting in:

- Faster application switching
- Lower latency
- Improved multitasking
- Higher overall throughput
- Better user experience

2. Cloud Server Memory Bottleneck

Problem

A cloud service provider experiences **high latency during database queries**. Database systems repeatedly access indexes, frequently used records, and query execution data.

The bottleneck may occur when required data is not available in cache or physical memory, causing expensive virtual-memory paging.

Cache Hierarchy vs Virtual Memory Paging

Feature	Cache Hierarchy	Virtual Memory Paging
Purpose	Speed up CPU data access	Extend available memory
Memory Used	L1, L2, L3 cache	DRAM and SSD/HDD
Speed	Very High	Slower
Managed By	Hardware	Operating System + Hardware
Unit	Cache line	Memory page
Main Problem	Cache miss	Page fault
Latency Impact	Moderate increase on miss	Very high when storage access occurs

Analysis

A cache miss causes the processor to search lower levels of memory.

However, when DRAM is insufficient:

Required Data



Not in DRAM



Page Fault



Storage Access



Very High Query Latency

If a database frequently swaps pages between DRAM and storage, query performance decreases significantly.

Recommended Memory Architecture

CPU



L1 Cache



L2 Cache



Large Shared L3 Cache



High-Capacity DRAM



Frequently Used Database Pages



NVMe SSD / Persistent Storage

Recommended Improvements

1. **Increase DRAM capacity**
 - Keeps more database pages in physical memory.
 - Reduces page faults.
2. **Use large L3 cache**
 - Improves access to frequently used indexes and query data.
3. **Use high-speed NVMe SSD**
 - Reduces storage delay when paging or loading data is unavoidable.
4. **Use database buffer caching**
 - Keep frequently accessed database pages in memory.
5. **Use NUMA-aware memory allocation**
 - Helps multi-processor cloud servers access local memory efficiently.

Result

This architecture provides:

- Reduced database query latency
- Fewer page faults
- Higher transaction throughput
- Better scalability for multiple users

3. Autonomous Vehicle Data Processing

Problem

An autonomous vehicle must process data from:

- Cameras
- LiDAR
- Radar
- GPS
- Other sensors

The system must make decisions in **real time**, so both low latency and high memory bandwidth are critical.

Comparison of SRAM, DRAM and MRAM

Memory	Speed	Capacity	Power	Best Use
SRAM	Very Fast	Low	Higher per bit	Cache and critical real-time data
DRAM	Fast	High	Moderate	Sensor buffers and AI models
MRAM	Moderate/Fast	Medium	Low standby power	Persistent critical data

Proposed Memory Hierarchy

Sensors

(Camera / LiDAR / Radar)



High-Bandwidth DRAM



AI / GPU / Accelerator



SRAM Cache / Buffers



CPU Cores



MRAM Persistent Memory

Design Explanation

1. SRAM

Use SRAM for:

- Frequently accessed sensor data
- Real-time control instructions
- Temporary AI computation results

SRAM provides very low latency, making it suitable for time-critical operations.

2. DRAM

Use high-bandwidth DRAM for:

- Camera frames
- LiDAR point clouds
- Large neural-network models
- Intermediate AI processing data

DRAM provides much larger capacity than SRAM.

3. MRAM

Use MRAM for:

- Critical system configuration
- Persistent control information
- Fast recovery after power interruption

MRAM is non-volatile, meaning it can retain data without continuous power.

Result

This design provides:

- Very low latency for critical decisions
- High bandwidth for sensor streams
- Large memory capacity
- Persistent storage for important information
- Improved safety and real-time performance

4. AI Inference Edge Device

Problem

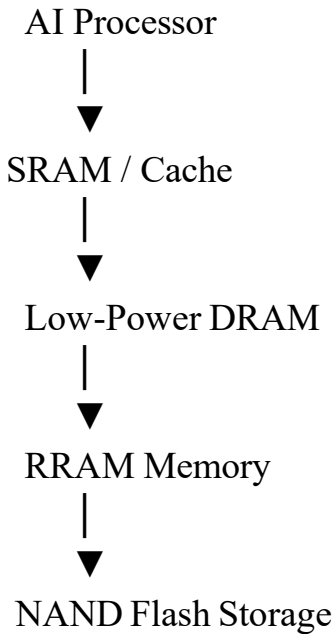
An edge AI device needs:

- Fast model loading
- Low power consumption
- Reduced memory latency
- Efficient local AI inference

Comparison of NAND Flash, DRAM and RRAM

Memory	Speed	Non-Volatile	Power Efficiency	Best Use
NAND Flash	Slowest	Yes	Good	Model and application storage
DRAM	Fast	No	Moderate	Active model execution
RRAM	Fast Potential	Yes	High	Persistent AI weights and edge computing

Proposed Memory Structure



Design Explanation

NAND Flash

Use NAND Flash for:

- Permanent storage
- Operating system
- Large AI models
- Application files

It provides large storage capacity at relatively low cost.

DRAM

When an AI model is being executed, the active parts of the model are loaded into DRAM.

DRAM provides faster access than NAND Flash.

RRAM

RRAM can be used for:

- Frequently used AI model weights
- Persistent AI parameters
- Low-power storage
- Fast model availability

Because RRAM is non-volatile, important model data can remain available even when power is removed.

Recommended Optimized Architecture

NAND Flash



Model Storage



RRAM



Frequently Used Model Weights



Low-Power DRAM



Active Model Layers



SRAM / Cache



AI Accelerator

Improvements

- Store complete models in NAND Flash.
- Keep frequently used model weights in RRAM.
- Load active model layers into DRAM.
- Keep immediate computation data in SRAM.
- Use intelligent prefetching to load the next model layer before it is required.

Result

This hierarchy provides:

- Faster AI model loading
- Lower energy consumption
- Reduced storage latency
- Improved inference speed
- Efficient edge-device operation

5. High-Performance Gaming Console

Problem

A gaming console experiences **frame drops while loading large scenes**.

Large scenes require:

- Textures
- 3D models
- Audio data
- Physics data
- Game world information

If the required data cannot be accessed quickly enough, the GPU or CPU may wait, causing frame drops.

L3 Cache vs DRAM Throughput

Feature	L3 Cache	DRAM
Location	Close to CPU	Main system memory
Latency	Low	Higher
Capacity	Smaller	Much larger
Throughput	High for cached working data	High bandwidth for large data sets
Best Use	Frequently reused data	Large game assets

Analysis

L3 cache is excellent for frequently reused information such as:

- Game logic
- Physics data
- Frequently accessed instructions
- Common object information

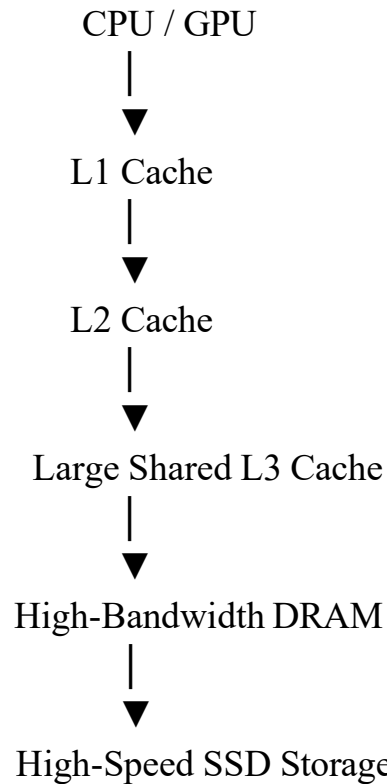
However, L3 cache cannot store an entire large game scene.

DRAM is necessary for:

- Large textures
- Character models
- Open-world data

- Large scene assets

Proposed Memory Hierarchy



Recommended Enhancements

1. Increase L3 Cache

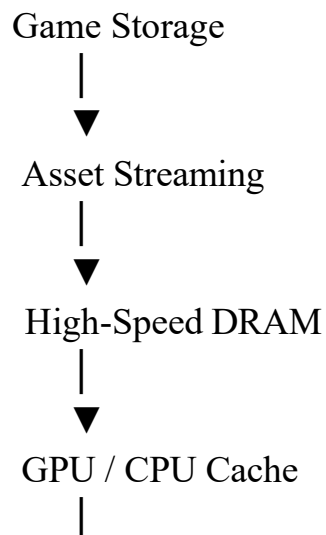
A larger L3 cache can store more frequently reused game data. This reduces repeated DRAM access.

2. Use High-Bandwidth DRAM

High-bandwidth memory allows large textures and scene data to be transferred faster.

3. Use Faster Asset Streaming

The system can load assets before the player reaches a new game area.





Screen Output

4. Use Intelligent Prefetching

The system predicts which game assets will be needed next and loads them early.

5. Compress Assets Efficiently

Compressed textures and game data reduce memory bandwidth requirements.

Final Result

The improved architecture provides:

- Reduced frame drops
- Faster scene loading
- Higher frame stability
- Better texture streaming
- Improved gaming performance

Overall Comparison and Recommended Memory Hierarchies

Scenario	Fastest Memory Level	Main High-Capacity Memory	Recommended Special Technology	Main Benefit
Smartphone	L1/L2 Cache	LPDDR DRAM	Shared L3 Cache	Faster app switching
Cloud Server	L1/L2/L3 Cache	High-Capacity DRAM	NVMe + Buffer Cache	Reduced query latency
Autonomous Vehicle	SRAM	High-Bandwidth DRAM	MRAM	Real-time processing
Edge AI Device	SRAM	Low-Power DRAM	RRAM + NAND	Fast, low-power inference
Gaming Console	L1/L2/L3 Cache	High-Bandwidth DRAM	Fast SSD Streaming	Reduced frame drops

Conclusion

The most effective solution in all five systems is to use a hierarchical memory architecture. Small and extremely fast memories such as L1 cache, L2 cache, L3 cache, and SRAM should store immediately required data. Larger memories such as DRAM should store active applications, sensor data, AI models, and game assets. Non-volatile technologies such as NAND Flash, MRAM, and RRAM can provide persistent storage and support fast recovery or efficient model storage.

The key principle is:

Frequently Used Data



Fastest Memory

Moderately Used Data



Cache / DRAM

Large and Persistent Data



Non-Volatile Storage

This approach minimizes latency, reduces bottlenecks, increases throughput, improves power efficiency, and delivers better overall system performance.